

2D Platformer

WebGPU Témalabor

Bevezető

Az én programom amit fejlesztettem, az egy kétdimenziós platformer játék pixel art stílussal. Megjelenítéshez a WebGPU API-t kellett használnom, aminek segítségével a grafikai rendering folyamatot lehet vezérelni. Habár alacsony szintű API-ról van szó, például a Vulkan-hoz képest egy magasabb szintű, könnyebben érthető dologról van szó, aminek még így is megvoltak a kihívásai.

A játék nem tartalmaz sztori konkrét elemeket, mint párbeszéd, vagy bármilyen írott formájú exozíció, inkább csak a megjelenés általi „environmental storytelling” fest képet arról, hogy miről is van szó. A játékos egy szellem alakját ölti fel, aki miután kiszabadult ketrecéből, a pokolból akar megszökni. Rövid pályákon át kell eljutnia a kezdőponttól a feljebb vezető ajtóig, fokozatosan nehezebb akadályokat leküzdve. Eközben az életét veszélyeztetik környezeti akadályok, mint láva, vagy démonok akik járőröznek, lőnek, vagy akár le is vadásszák a játékost. Végző cél kijutni az összes szinten át a felszínre, és visszajutni a szellem saját sírjához, ahol a poklon kívül nyugalomban tud megpihenni.

A játékos, egy 2D platformerhez hűen képes balra és jobbra mozogni, és ugrani. Illetve, a játékos el is tudja „fektetni” a karakterét, ami megjelenés szinten olyasmit jelent, hogy a szellem szétesik egy kupac ektoplazmára, ami széles, ámbár jóval alacsonyabb mint teljes formában. Ebben a formában kisebbet tud ugrani, lassabban esik, és akármikor tud váltani a teljes és szétesett forma között, feltéve, hogy érintkezik a talajjal. Ennek a mechanikának köszönhetően több fajta pályát lehet készíteni, hiszen a két forma más-más dolgokra képes. Teljes formában 2 egész kockányit tud ugrani, szétesve csak egyet. Szétesve 1 kocka magas alagútban is tud ugrani, teljes alakban nem, mivel a szellem pont 1 kocka magas alaptól, szóval beveri a fejét. Előfordulnak olyan szintek, ahol a kijutáshoz többször is alkalmazkodni kell az adott helyzethez a formaváltással, olykor úgy is, hogy egy démon a játékos nyakán van végig.

Fejlesztés

A játék első fázisaiban még a játék aspektus sehol sem jelent meg, ugyanis először a WebGPU rendereléssel kapcsolatos alap dolgokat vittem be a programba. A WebGPUfundamentals oldal és egyéb dokumentáció segítségével először egy sima háromszöget rajzoltam ki a képernyőre. Itt a háromszög pontjai a WGSL vertex shader-ben voltak hard-codeolva, szóval bármilyen változtatáshoz még oda kellett belenyúlni. A színezést csak egy egysoros fragment shader oldotta meg, szimplán csak lilára színezve minden pixelt. Egy kisebb refaktor után a következő lépés az volt, hogy quad-ok renderelésére állítottam, ami csak kettő darab háromszög összetéve, hogy egy négyszöget adjon ki. Én kifejezetten szabályos téglalapokat és négyzeteket terveztem használni a játékhoz, de akár előfordulhatna nem szabályos alakzat is. Amire már itt figyelmet fordítottam, az az volt, hogy a quad renderelés az képességeimnek megfelelően optimális legyen. Alap esetben kettő háromszög az logikusan hat darab csúcsot jelentene. Viszont, a négyszögeknek nevéből adódóan csak négy csúcsra van szükségük, tehát feleslegesen pazarolnánk számítási kapacitást arra, hogy mind a hat csúcsot megjelenítsük. Erre a megoldást index buffer használata jelentette, amivel így a két háromszög rendesen osztozni tudott azon a két csúcson ami mindkettőhöz tartozott. Ezen kívül, a háromszögek csúcsait már nem a shaderbe beégetve tároltam, hanem a TypeScript kódban definiáltam őket, és küldtem fel a GPU-ra vertex shader-ben. Így ezzel már elkészültek a renderelés fő alapkövei, amiket még a textúrák előtt sokat használtam.

Következő iterációban TypeScript interface formájában felvettem téglalapot és pályát, és funkcionalitást, amely a téglalapok X, Y, szélesség, magasság tulajdonságait normalizálják a GPU számára. Mivel csak kétdimenziós renderelésről van szó, statikus nézettel, ezért a Z térbeli vagy a W homogén koordinátákkal való bármilyen fajta számolást nem végez a program. A Z értéke 0, W értéke 1, és ezekkel a konstans értékekkel tökéletesen működik. Vagyis, csak egy tömböt szimplán megtöltve X, Y, W, H, és szín tulajdonságokat tartalmazó objektumokkal, az egész pályát meg tudjuk jeleníteni.

Ezután a játékost vezettem be mint osztályt, ami igazából sok mindenben nem különbözött itt még egy sima téglalaptól.

Szélessége és magassága konstans, szóval alakja mindig szabályos. Egy alap gravitáció implementáció is ekkor került a programba, ami csak tized másodpercenként növelte a játékos Y koordinátáját (a képernyőn lefele nő az Y koordináta), addig, amíg az aljára nem ért. Ezzel természetesen a fő probléma, hogy semmit nem vesz figyelembe, csak a képernyő alsó határát, tehát más téglalapokkal semmi kölcsönhatás nem volt.

Mivel minden téglalap tökéletesen illeszkedik az X és Y koordinátákra, vagyis semmilyen elforgatás nincsen, ezért az ütközések detekciójára jó megoldást nyújtott az AABB algoritmus. Ennek a lényege, hogy végrehajtjuk a mozgást az objektumok sebességeivel, és az utána kialakult állapot alapján döntjük el, hogy szükséges-e beavatkozás, és milyen formában. Ha a két téglalap ütközik, azt úgy tudjuk eldönteni, hogy megvizsgáljuk mindkét téglalapot, és mindkét koordináta mentén megnézzük, hogy van-e átfedés. Ha csak az egyik dimenzióban fedik egymás az nem elég, mivel az jelentheti például azt, hogy ugyanabban a magasságban fekszik a földön két téglalap, de egymás mellett, érintkezés nélkül. Ütközés esetén megkeressük, mi a legkisebb vektor amivel elmozgatva a játékost az ütközés megszűnik. Ez a vektor tisztán vagy vízszintes vagy függőleges.

Ezzel az egyik legnagyobb probléma ami előjön, az az, hogy a legrövidebb út kifelé, az nem mindig a leglogikusabb út is egyben. Ha a játékos zuhan, és egy téglalap legszélére ráesik, akkor esési sebességtől függően valahány pixel mély függőleges ütközés történik. De mivel a legszélére esett rá, ezért a vízszintes mélység az csak pár pixel lesz, vagyis az a rövidebb út kifelé. Ez szemmel nézve olyan, mintha ráesnénk egy platform szélére, és csak lecsúszunk róla oldalirányba.

A probléma orvosolására az első megoldás az volt, hogy amikor kiválasztjuk az ütközés megszüntetéséhez a vektort, akkor ha a játékos esik és nem mozog, akkor preferáljuk a függőleges vektort, még akkor is ha a vízszintes rövidebb. Ezen kívül annak eldöntésére, hogy a játékos a földön áll-e, azt a megoldást találtam, hogy a játékos alá egy 1 pixeles láthatatlan téglalapot projektálunk. Vagyis, ha az a téglalap ütközik valamivel, akkor a játékos valamin áll. Továbbá a JavaScript eventjeit használva alap input kezelést is bevezettem mozgásra.

Mindezek után ha csak simán esik a játékos akkor semmilyen lecsúszásos probléma nem merült fel, viszont az esés és mozgás kombinációja az még így is elő tudta idézni a problémát, habár egy picit nehezebben volt észrevehető.

Ami viszont még talán ennél is zavaróbb probléma volt, az a „tunneling” effektus. Ez is az AABB egyik sajátosságából adódik, abból, hogy először végrehajtjuk a két időpillanat közti elmozdulásokat, és csak a következményekkel foglalkozunk. Ugyanis, ha a játékos pillanatnyi sebessége valamelyik irányban nagyobb, mint az útjában álló akadály vastagsága (a mozgás irányában nézve), akkor egyik pillanatról a másikra a játékos csak áthalad a másik objektumon, mintha ott sem lett volna. A működés logikus, hiszen a játékost reprezentáló téglalap szempontjából sosem volt ütközés semmivel, de ránézésre látjuk, hogy mégiscsak nem elvárt működés az, hogy a falon / padlón csak átmegyünk. Bár nem csak akkor jelenhet meg a probléma, ha teljesen áthaladunk az objektumon, hanem akkor is, ha csak eléggé behatolunk ahhoz, hogy a másik oldal utána közelebb legyen, mint az eredeti oldal. Erre megoldást a sima AABB algoritmus lecseréléséig nagyon nem lehet találni, ezzel jár. Elkerülni el lehet, ha a játékos sebessége nem nőhet bizonyos értékeknél nagyobbra. Az hogy ez mekkora, ez függ attól, hogy milyen széles az adott téglalap aminél előfordulna a tunneling, a behatolási irány, és a játékos mérete / magassága, attól függően, hogy melyik irányból közeledünk. Például egy 50 pixel vastag falat balról közelítve, ha a játékos 30 pixel széles, onnantól van rá esély arra, hogy tunneling lépjen fel, ha a sebessége jobbra nagyobb, mint $(30 + 50) / 2 = 40$.

Itt vezettem azt a mechanikát is, hogy a játékos el tud „feküdni”, ami lényegében csak annyit jelentett, hogy kicserélte a szélességet a magassággal. Ennek is megvoltak az érdekes bugjai eleinte. Ha csak simán megcseréljük a kettőt, és a játékos XY koordinátáit nem korrigáljuk, akkor ismét tunneling jött elő. Mivel a játékos szélessége csak megnőtt jobbra, ezért a téglalap közepe is jobbra csúszott. Ha egy fal mellett feküdt le, akkor belelőgtünk egy pillanatig a falba, és ha az új közép a fal másik oldalához volt közelebb, akkor az AABB arra dobta ki a játékost. Ezért kellett a játékos lefekvés utáni helyzetét úgy beállítani, hogy a közepe se jobbra, se balra ne mozduljon el. Itt sem volt még vége a problémáknak azonban. Ha a fal mellett a levegőben lefeküdtünk, akkor egy pillanatig ahogy belecsúszott a falba a játékos, úgy érzékelte, hogy talajon áll, ezért a gravitáció nulláról indult újra.

Egy iterációban kísérleteztem azzal megoldással is, hogy két téglalapot használni a játékosnak, amiből az egyik 1 pixellel nagyobb – ez lett volna megjelenítve, és egy másik „alatta” – ez lett volna a hitbox. De hamar inkább elvettem az ötletet, felesleges extra bonyolultságnak tűnt, és a szoros érintkezés okozta problémát orvosoltam máshogy.

Eddig a pontig a játék fizikáját vezérlő függvény csak meg volt hívva egy `setInterval` függvénnyel az oldalon, ami azt jelentette, hogy a renderelés és a logika között nem volt egymás utáni sorrend. Előfordulhatott például az, hogy olyan képkocka jelent meg, ahol a játékos félig a falban van, mielőtt az AABB algoritmus sikeresen kitolta volna onnan. Ezért vezettem be a játékokban sokszor használt delta időes megoldást. Ez azt foglalja magában, hogy mindig, amikor a frame függvény meghívódik, megkapja a jelenlegi időt, és összehasonlítja a legutóbbi hívás időpillanatával. Ha ez a különbség nagyobb, mint 16,67 milliszekundum (a másodpercenkénti 60 lefutáshoz szükséges időkorlát), akkor egyszer végigmegy a fizikát vezérlő függvény. Ha nem, akkor nem történik semmi. Ettől függetlenül a renderelés megtörténik.

Ezután áttértem arra, hogy pár mechanikát hozzáadjak a játékhoz mielőtt a collision problémáimhoz visszatértem. Először is, a játékos elfekvése most már stabilan működött, de a fekvésből való felállás kisebb problémát tudott okozni. Ha a játékos egy olyan alacsony helyre kúszott be, ahová állva nem tudott volna, akkor ha ott felállt, beleragadt a felette lévő téglalapba. Könnyű volt erre a megoldás, ha a játékos feláll, és annak következtében bármilyen fajta ütközés következik be, akkor egyből vissza is fekszik. A játék kész változatában ez nem tud előfordulni, mivel a játékos pont olyan magas, mint egy tile, szóval annál kisebb alagút nem létezhet, de ezt ekkor még nem láttam előre. Ezután a szintek változtathatóságaért felelős kódrészletet készítettem elő, eddig csak simán bele volt hard-codeolva, hogy melyik szint megy.

Az ugrás funkciót is implementáltam, eddig azt sem lehetett. Itt nem vittem túlzásba a dolgot, ugrás pillanatában a játékos kap egy löketet a függőleges sebességére, és utána a gravitáció megoldja a lassulást, és végül a visszaesést. A játékoson pedig állapotok tartoznak az akciókhoz, mint mozgás, ugrás, esés, és ezek diktálják, hogy egy gombnyomás mit csinál. Például ugrás közben megint ugrani nem lehetséges.

Végül pedig az ugrást egészítettem ki azzal, hogy ahhoz hasonlóan, mint amikor a talajjal való érintkezést csekkoltam, a plafonnal is ugyanezt megcsináltam. Erre azért volt szükség, hogy amikor a játékos beveri a fejét ugrás közben, akkor ne csak ott lebegjen a plafonon mielőtt a gravitáció eléggé lehúzza, hanem egyből kezdjen el visszafelé esni. Egy probléma volt még az ugrással, hogy az ugrás állapot megszüntetését ahhoz kötöttem, hogy a játékos még nyomja-e a gombot, vagy sem. Ezzal az volt a baj, hogy ha földet ért, de még mindig nyomta, akkor zuhanó állapotban volt, a föld felé gyorsulva, amíg el nem engedte a gombot. Ezért ezt csak szimplán átszerveztem, hogy amikor a gravitáció nullára állítja a sebességet, akkor vegye le az ugrás állapotot is.

Ilyenkorra már egy egészen használható játék alapjait kaptam, ami az AABB-től eltekintve egész jól működött.

A következő, nagy változtatás a sima AABB algoritmusom helyett a továbbfejlesztett, Swept AABB algoritmus implementálása volt. Az eredeti implementációhoz képest ez abban különbözik, hogy nem hajtjuk végre egyből a mozgást, hanem előre megnézzük, hogy a mozgás következtében mikor következne be ütközés. Itt már „időben” gondolkodunk, de ez igazából csak az éppen esedékes tick „időtartamát” jelenti. Minden pályán lévő téglalapnál megnézzük, hogy ütköznie kéne-e valamikor, ha mozognánk abba az irányba a mostani sebességgel. Ha sehol nincs ütközés, akkor csak a sebességének megfelelően elmozdul a játékos, de ha például a kezdőpont és végpont között félúton van egy fal, akkor a mozgás csak a kijelölt időtartam feléig fog tartani. Ha van egy még közelebb, egyharmados távban, akkor az lesz a nyerő. Mindig a legrövidebb ütközési távolság mondja meg meddig szabad mozogni. És ennek következtében a játékost pont annyival mozdítjuk csak el, hogy pont 0 pixel legyen a távolság közte és a fal között amibe ütközött volna. Ezentúl, az ütközési felületre merőleges sebességet kinullázza.

Itt az algoritmusom már lényegében hibátlanul kellett volna működjön, a célnak megfelelően legalábbis, de olyan probléma jött elő, hogy néhány esetben nem tudott a játékos lemozogni egy platformról, mert megakadt pont az utolsó pixelen, amikor a két téglalap sarka ütközött. Ekkor még ez nem tudtam, hogy ez szimplán egy olyan kis hiba miatt lépett fel, hogy reláció vizsgálatnál ' $<$ ' helyett ' $<=$ '-t írtam.

Mivel még nem sikerült rájönnöm mi volt a hiba, ezért bevezettem egy kis delta értéket, ami azt mondta meg, hogy ha egy nagyon apró számnál kisebb az ütközési időpont, akkor azt hamis pozitívnak vegye. Ez esésnél működött is, és most már a sarkokon sem akadtam meg, viszont ha egy falnak nekimentem, akkor azon egy az egyben csak át tudott menni a játékos, mivel az ütközési időpont a falnak simulva 0, vagyis a deltánál kisebb volt. Ezért itt csak visszaírtam a sima AABB függvényhívást, és mindkettőt egyszerre használva ment tovább a fejlesztés.

Tovább haladva, a pályák közti váltást valósítottam meg. Ez a funkció annyit tesz, hogy az egyik téglalap ki van nevezve „ajtónak”, és ha ehhez hozzáérünk, akkor a következő szintre csak átvált a játék, ha létezik. Az ajtó átjárható, vagyis nem falként viselkedik. Következőnek látát implementáltam, vagyis olyan téglalapot, aminek ha nekimegyünk, akkor a pálya az elejéről indul. Ehhez egy callback function-t használtam, azt adtam át a fizikát vezérlő függvénynek, amit egy tömb téglalapra meghívva megnézi, hogy valamelyik láva-e, és ha igen, akkor megöli a játékost. A callback-re sok szükség nem volt, de így a separation of concerns-t jobban követte. És, így könnyen lehetne bővíteni azt a callback-et más esetekkel is, amik más funkciót töltenének be, például, ha valami collectible tárgynak a felvételét kellene lekezelni.

Következőnek implementáltam a textúrás pipeline-t. Eleinte minden quad-ot csak átraktam textúrásra, és ugyanazt a textúrát töltöttem rá mindenre, szóval nehéz volt bármit is látni a játékból, de a játékos mozgó alakját ki lehetett venni. A textúra 100x100-as volt amit itt használtam, ezért bevezettem egy 100-as csempézést csak textúra szinten, vagyis egy 200 pixel széles, 100 pixel magas téglalap a textúrát nem kétszer olyan szélesre húzta szét és jelenítette meg, hanem kétszer egymás mellé a 100x100-as négyzetet. Minden aminek a mérete nem illeszkedett a 100 valamilyen többszörösére, az viszont nem jelent meg szépen. A játékos téglalapja például, mivel kicsi, ezért megjelenése csak egy kis hasáb volt az eredeti textúrából. De ez volt a helyes működés ebben az esetben, ezt is vártam. Legfőbb probléma itt a textúrák összeillesztésénél volt, mivel ha két téglalap között az eltolás az nem illeszkedett erre a 100 pixeles csempézésre, akkor a textúrákban ez nagyon feltűnő volt.

Legalább, hogy lássam mi történik, megcsináltam, hogy a textúrázott, és a szimplán színes rendering pipeline is egyszerre jelen legyen a programban, és a téglalapok textúra attribútumának létezése alapján döntöttem el render időben a CPU-n, hogy melyik vertex bufferbe kerüljenek az adatai, és hogy melyiknél melyik pipeline-ra kell átállni mielőtt kirajzolja annak a téglalapnak a csúcsait.

Ezután refaktoráltam a rendering folyamatot egy kicsit, és lehetségessé tettem az áttetsző textúrák használatát is, hogy ha egy textúrán nincs minden pixel beszínezve, akkor az alatta lévő textúrát / színt jelenítse meg.

A tematika a fejlesztés első fázisaiban még nem tudtam mi lesz pontosan. Még mielőtt textúrákat vezettem volna be, a játék minimalisztikus megjelenése kifejezetten tetszett, olyan játékokra emlékeztetett, mint a Level Devil, vagy a VVVVVV. Most már viszont el kellett gondolkodnom, hogy milyen kinézetet szeretnék a játékomnak úgy igazán. Az Animal Well kinézete inspirálta a folyamatot ahogyan elkészítettem az első textúrámat, egy pirosas, lilás, téglaszerű elemet. Azzal a szabállyal kezdtem el innentől mindegyiket rajzolni, hogy kisebb felbontásból négyszeresen felnagyítottam őket, szóval ami 1 pixelnek néz ki szemmel, az igazából 4x4 pixel a textúrán. A játékos kinézete végül egy szellem lett, mivel úgy gondoltam, hogy ennél nem néz ki zavaróan az animáció hiánya, és belemagyarázható az 'elfekvés' mechanika, azzal, hogy a szellem mivel nem egy szolid dolog, ezért szét tud esni ekto plazma darabokra majd újra összeállni. A láva ötlet már alaphoz megvolt, ezért ezekkel a meglévő textúrákkal eldöntöttem, hogy a helyszín az lényegében a pokol lesz, majd később, pedig a pokolból való megszökést jelöltem ki, mint célt.

Bevezettem azt is, hogy egy textúrát lehessen eredeti irányban, vagy vízszintesen tükrözve kirenderelni, ami az objektum egyik attribútumától függ. Ezzel már a szellem el tudott nézni jobbra, ha jobbra megyünk, vagy balra, ha arra fordulunk.

Következőnek ellenfeleket implementáltam, aki nem sokkal később kapott egy lebegő démon fej textúrát, a Doom Cacodemon-járól mintázva. A démon funkcionalitása az, hogy egy kezdő és egy végpont között ingázik oda-vissza, és ha a játékos nekimegy, akkor meghal. Lényegében egy mozgó akadály volt még ebben az iterációban.

Következő dolog a napirenden az volt, hogy a játéknak tényleg vége is legyen. Eddig ha az utolsó pályán nekiment a játékos az ajtónak, akkor csak nem történt semmi, és kész. Beállítottam, hogy ilyenkor egy flag-et beállítson, hogy a játéknak vége, és ez egy áttetsző HTML elemet jelenített meg a játék fölött, meg szöveget. Az R betű megnyomására pedig újraindult az egész. Ezzel voltak kisebb problémák amiket a fejlesztés során folyamatosan küszöböltem ki, amikor észrevettem őket. Minden pályának saját játékos objektuma van, és a pályák közti váltás csak azt mondja meg, hogy melyik pályára kerül a „fókusz”. Azt jeleníti meg, ott mozognak a szörnyek, ott mozog a játékos. Szóval váltás után azon a pályán amit elhagytunk, a játékos az ott maradt ahol végeztünk vele, abban az állapotban, és azokkal az attribútumokkal. Tehát játék újratekésztésénél minden pályán meg kellett ölni a játékost, hogy alaphelyzetbe kerüljön, és minden ellenfelet is vissza kellett állítani kezdőpozíciójára.

Ezután megkezdtem az egész pályát átszervezni csempés megoldásra, hogy elkerüljem a kellemetlenségeket a pályák szerkesztésénél, és a textúráknál, amikor nem jön ki pont jól a téglalapok méretével. 64x64-es csempék mellett döntöttem, az ajtó 32 széles, a szellem pedig 24 széles lett. Az egész pálya 14x14 csempéből áll. Ezzel a szervezéssel a pályák létrehozása is sokkal egyszerűbbé vált, mivel így már egy nagy string-et be tudtam parse-olni és abból kialakítani a játszható pályát. A játék kész verziójából egy ilyen string például így néz ki:

```
BBBBBBBBBBBBBB
BLLLBSCSBSB+_B
BBBB_BBB_BBB_B
-----B
-----B
B__BBBBBBBBBB
BB__BXXXX__X
BBB_B_____X
BBB_____BX_X
BBB_B_____B_X
BBBLBLLLLBB_BB
B_____B
B#_____
BBBBBBBBBBBBBB
```

Minden karakternek meg van adva a saját jelentése, és ezek alapján a konstruktor a mátrix megfelelő helyére a megfelelő elemet rakja. Például a '+' az a kezdő pozíció, a '#' a cél, 'L' az láva, 'B' az sima téglás csempe. Sajnos a járőröző démonokat külön kell a konstruktorban megadni, mivel ez a string nem adna minden opcióra megoldást, de így is nagy pozitív hatása volt ennek az új rendszernek.

Következőnek a rendering ciklust refaktoráltam, mivel sok felesleges számolást végeztem el képkockánként, aminek semmi értelme nem volt. A vertex buffereket különválasztottam statikus és dinamikus bufferekre, és a statikus buffereket nem képkockánként, hanem csak akkor készítettem el újra, amikor megváltozott a szint. A dinamikus buffert mindig újratölti a játék, de az a pályától függően sokkal kevesebb adatot tartalmaz, és ezt muszáj is megcsinálni mindig, mert a dinamikus dolgok framek közt változhatnak.

A tile rendszerrel kapcsolatos első bug az volt, hogy minden nem üres csempét beleszámítottam az ütközésetektálásba, ami jelen állás szerint azt jelentette, hogy megáll a szellem a láva tetején, mintha sima csempe lenne. Ezt azzal kellett megoldanom, hogy nem mindig a teljes csempe listát kérem le, hanem funkciótól függően csak a nem halálos csempéket.

Következőnek a démont bővítettem opcionális lövés funkcióval, amellyel fel, le, balra, vagy jobbra tud halálos lövedékeket lőni magából paraméterezhető intervallumonként. Eleinte itt ez az intervallum is a JS setInterval függvényét használta, ami később azt eredményezte, hogy a játék tick-jeitől el tud csúszni.

Mivel a csempés rendszer már teljesen használhatóvá vált, ezért ezt a mátrixot gráfként használva egy útkeresésre képes démont is tudtam implementálni, A* algoritmussal. Mivel mozgás közben a démon két csempe között is lehet átjáróban, ezért ilyen állapotban semmiképpen nem akartam, hogy irányt váltson, ezért új útvonalat csak akkor deríthet fel, amikor eléri a céljához vezető következő csempének a közepét.

A korábban említett relációs hibát kijavítva most már elég volt önmagában használni a Swept AABB-t a sima AABB nélkül.

Bevezettem egy saját input-kezelő rendszer ami helyesen tudja kezelni a több, egyszerre lenyomott gombot, de ez azzal járt, hogy a Swept AABB elromlott. Minden frame-ben újra tudjuk frissíteni a mozgási szándékunkat, és ha ez egy falba vinne, akkor a Swept AABB kinullázza a mozgásunkat abban a frame-ben. Szóval falnak menve a levegőben, rá tudtunk tapadni, mint Pókember. A megoldás erre az lett, hogy szétválasztottam az algoritmust egy vízszintes és függőleges részre amik egymás után futnak, így annak ellenére, hogy a vízszintes komponens kinullázódik, a függőleges nem.

Következőnek megcsináltam, hogy ha a játékos kimegy a pálya keretein kívülre, akkor meghal. De balra és jobbra kimenni és egyből meghalni kicsit túl kellemetlennek tűnt, ezért bevezettem a pálya bal és jobb szélére a képernyőn kívül 1-1 oszlop csempét, ami blokkolja a mozgást. Ezzel az volt a baj, hogy logikusan a játékos sor, oszlop pozíciója nem kellene megváltozzon, de mivel létezik a mátrixban egy extra üres csempe minden sor legelején, ez nem teljesen volt így igaz. Ez aztán problémákat okozott az útkeresésben és egyéb függvényekben, amiket sikerült megoldani.

A lövésre képes démont a setInterval-ról áthelyeztem egy belső számlálóra ami a játék tick-jeit használja időzítésre, és így mindig konzisztens eredményeket kaptam. Ezentúl opcionális késleltetést is bevezettem az első lövés előtt, hogy komplexebb akadályokat is létre lehessen hozni.

Legvégül refaktoráltam sok osztályt, a fizikát vezérlő modult is osztályba áthelyeztem, próbáltam mindent egy átláthatóbb struktúra alá helyezni. Nem emeltem ki külön minden textúra elkészítését, de fejlesztés közben folyamatosan adtam hozzá pár újat, amikor szükségesnek láttam, és így 13 darab textúra van a legvégső formában, amiből 2 textúra az textúra atlasz, ezt használom dinamikus textúrázásra szomszédok alapján. Elkészítettem több pályát is, összesen tízet. Az első 5 az könnyű, tanító jelleggel, a következő 4 viszont jóval nehezebb. Az utolsó lényegében kihívást nem tartalmaz, csak egy levezető – a szellem visszatér a sírjába a felszíni világban.



